It is people that require software to change, not the software itself, which I touched upon in the previous section when describing problem domains.

Consider the final example from the previous section: the `StockTaker`. This hypothetical class existed to query shop and warehouse inventories to calculate the remaining amounts of books. Likely its output will be used by other classes, perhaps to generate reports or financial forecasts. The reporting and forecasting functionality could be written directly into `StockTaker`, with a design like this:

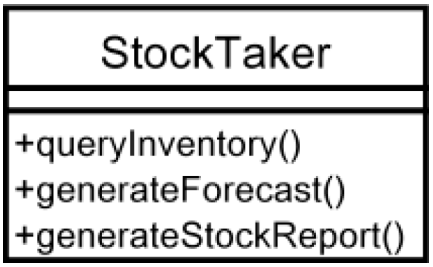


Figure 10-12. *`StockTaker` class design*

Now think about the stakeholders for this class, all of the people within the business that have an interest in what it does and, crucially, might one day need it to change. Finance wants the forecast, warehouse managers want the stock report, and so on. To say nothing of the rest of the code in the system that has a dependency on `StockTaker`. In order to isolate interested actors, we need to change the design.

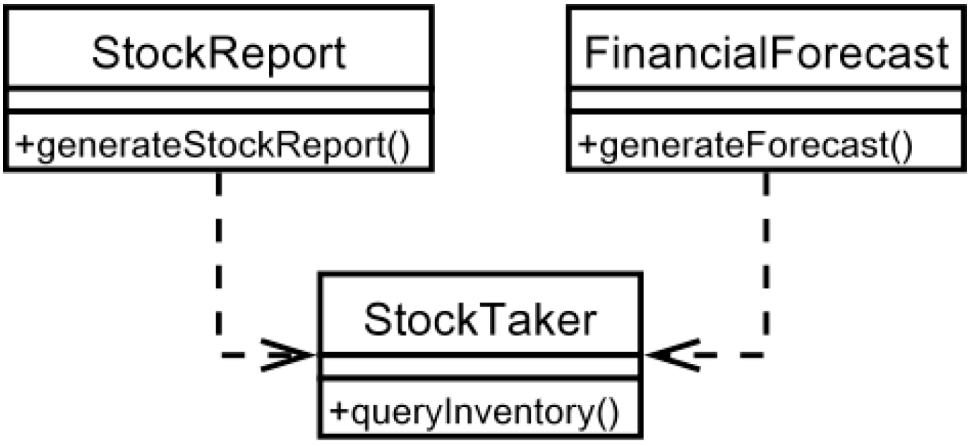


Figure 10-13. *Isolating stakeholders by design*